

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

Introducción	04
Componentes del DFD	08
Diferencias entre el DFD y el Diagrama de Contexto	11
Bibliografía	24

Clase 3



¡Te damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- ¿Qué es un Evento?
- Tipos de Evento que existen.
- Qué es un Diagrama de Flujo de Datos (DFD).
- Diferencias entre el Diagrama de Contexto y el DFD.
- La importancia de la Experiencia de Usuario (UX).
- Condicionales, manipulación de cadenas y bucles en Python.

Introducción

En esta clase abordaremos dos conceptos esenciales para el análisis y modelado de sistemas: los eventos y el Diagrama de Flujo de Datos (DFD).

Estos conceptos permiten comprender cómo un sistema responde a estímulos, ya sean internos o externos, y cómo la información se transforma y circula dentro de él.

Antes de avanzar, es importante retomar brevemente el concepto de scoping, introducido en la clase anterior. El scoping nos permite definir los límites del sistema: qué procesos, actores y funcionalidades están incluidos en el análisis y cuáles quedan fuera. Esta delimitación resulta fundamental para identificar las entidades externas que interactúan con el sistema, así como los eventos que desencadenan acciones dentro del mismo.

Comprender los eventos y su representación mediante DFD es esencial para modelar el comportamiento del sistema de manera precisa y efectiva, lo cual facilita la comunicación entre analistas, desarrolladores y usuarios finales.

Habiendo introducido y refrescado los conceptos aprendidos en la clase pasada, es importante pensar en los eventos, pero, nos hacemos primero la siguiente pregunta:

¿Qué es un Evento?

Un evento es cualquier suceso, estímulo o condición que provoca que el sistema realice una acción o genere una respuesta. Puede considerarse como un "disparador" que pone en marcha un proceso dentro del sistema.

¿Por qué son importantes los eventos?

- Definen el comportamiento dinámico del sistema.

- Permiten identificar qué situaciones deben ser gestionadas por el sistema.
- Constituyen la base para construir la lógica del sistema.
- Ayudan a determinar las entradas y las condiciones bajo las cuales deben ejecutarse determinadas funciones.

Tipos de eventos

Eventos externos

Son aquellos que se originan fuera del sistema, generalmente por parte de una entidad externa como un usuario, otro sistema o un dispositivo físico.

Ejemplos:

- Un cliente realiza un pedido desde una página web.
- Un usuario presiona un botón para solicitar un informe.
- Un sensor envía una señal indicando que se superó un umbral de temperatura.

Características:

- Son impredecibles en cuanto a su ocurrencia.
- Constituyen la principal vía de interacción entre el sistema y su entorno.

Eventos temporales (o internos programados)

Son eventos que ocurren de forma automática o periódica, sin intervención directa de un actor externo en el momento de ejecución. Estos eventos permiten al sistema realizar tareas rutinarias o controles autónomos.

Ejemplos:

- Un sistema de inventario que revisa automáticamente el nivel de stock cada 24 horas.
- Una aplicación que envía recordatorios semanales a los usuarios.

- Un servidor que realiza copias de seguridad diariamente a la medianoche.

Características:

- Son predecibles y repetitivos.
- Su ejecución es controlada por el sistema o un temporizador.

Los eventos son fundamentales para construir tanto la lista de acontecimientos como para modelar la lógica de funcionamiento del sistema mediante diagramas.

Diagrama de Flujo de Datos (DFD)

El Diagrama de Flujo de Datos es una herramienta gráfica utilizada para representar el flujo de información dentro de un sistema.

Muestra:

- La procedencia de la información (orígenes externos).
- Cómo se transforma o procesa dicha información.
- Su destino final (almacenamientos o salidas).

El DFD permite representar los procesos sin necesidad de describir decisiones o estructuras de control, por lo que es especialmente útil en la fase de análisis.



Elementos existentes dentro de un DFD - Fuente: [Diagrama de Flujo de Datos](#)

Además, permite representar procesos sin necesidad de mostrar decisiones o estructuras de control, por lo que resulta especialmente útil durante la etapa de análisis.

¿Por qué usar un DFD? Ventajas:

- Facilita la comprensión del funcionamiento interno del sistema.
- Permite visualizar las interacciones entre procesos, datos y actores externos.
- Ayuda a detectar redundancias, cuellos de botella y oportunidades de mejora.
- Sirve de puente entre la especificación de requisitos y el diseño técnico.

Ejemplo: Al generar una factura, el código de la misma se crea automáticamente tomando como referencia el número de la última factura emitida.

Relación entre eventos y DFD

- Los eventos actúan como disparadores de procesos dentro del DFD.
- Un evento externo se representa como una entidad externa que envía datos a un proceso.

- Un evento temporal puede activar un proceso automático dentro del sistema, representado como un proceso interno.
- En conjunto, permiten modelar el comportamiento del sistema desde el estímulo inicial hasta la respuesta generada.

Componentes del DFD

El DFD se compone de cuatro elementos principales:

Procesos: Transforman datos de entrada en salidas. Representan actividades o funciones del sistema. Se ilustran con círculos o burbujas y deben contar con al menos un flujo de entrada y uno de salida. Se numeran para su identificación (ej.: 1.0, 2.0).

Flujos de datos: Indican el movimiento de la información entre procesos, almacenes y entidades externas. Se representan mediante flechas que muestran la dirección del flujo, y su nombre debe describir el contenido del dato.

Almacenes de datos: Lugares donde se conservan datos para ser utilizados posteriormente. No implican necesariamente bases de datos digitales; pueden ser planillas, archivos físicos, etc. Se dibujan como dos líneas paralelas o un rectángulo abierto en uno de sus lados.

Entidades externas (o terminadores): Son actores que interactúan con el sistema pero que no forman parte de él. Pueden ser personas, organizaciones o sistemas externos. Se representan con un rectángulo.

Veamos en detalle cada uno de ellos:

a) Proceso

Representan actividades o funciones del sistema.



b) Flujo de datos

Representa el movimiento de datos entre procesos, almacenes y entidades externas.

- Se dibuja como una flecha.
- La dirección indica el sentido del flujo.
- El nombre debe reflejar el contenido del dato que circula.



c) Almacén de datos

No necesariamente implica una base de datos informática; puede referirse a cualquier forma de persistencia.

- Se representa como dos líneas paralelas o un rectángulo abierto por uno de sus lados.

Ejemplos: archivos físicos, planillas, sistemas externos, bases de datos.



Importante: A nivel de análisis, el almacén de datos se considera de forma abstracta. Su implementación técnica concreta (por ejemplo, como base de datos relacional) se decide posteriormente durante el diseño.

d) Terminador o entidad externa

Representa una entidad que interactúa con el sistema pero que no forma parte de él. Puede ser una persona, un sistema externo u otra organización.

- Se representa mediante un rectángulo.
- Intercambia datos con el sistema a través de flujos de entrada y salida.



Ejemplo:

Sistema de Gestión de Pedidos

Evento externo: Un cliente realiza un pedido online → esto es un estímulo que el sistema debe capturar y procesar.

Evento temporal: El sistema verifica diariamente si hay pedidos pendientes → proceso automático que puede activar notificaciones o reportes.

En el DFD:

- El cliente es una entidad externa que envía datos (pedido) a un proceso "Registrar Pedido".
- El proceso "Registrar Pedido" almacena la información en una base de datos.
- El evento temporal dispara el proceso "Verificar Pedidos Pendientes".

Diferencias entre el DFD y el Diagrama de Contexto

Característica	Diagrama de Contexto	Diagrama de Flujo de Datos (DFD)
Propósito	Mostrar una visión general del sistema como una sola unidad.	Mostrar el funcionamiento interno del sistema mediante procesos y flujos de datos.
Nivel de detalle	Muy bajo (alta abstracción).	Mayor nivel de detalle, con múltiples niveles posibles.
Procesos mostrados	Solo un único proceso que representa todo el sistema.	Múltiples procesos interrelacionados que representan funciones específicas.
Entidades externas	Sí, se identifican y se muestran todas las entidades externas relevantes.	También se muestran, pero en función de los procesos que interactúan con ellas.
Almacenes de datos	No se incluyen.	Sí, se representan explícitamente.
Finalidad	Delimitar el alcance del sistema .	Entender cómo circula y se transforma la información dentro del sistema.
Representación	Un círculo o rectángulo con el nombre del sistema y flechas de entrada/salida.	Múltiples procesos conectados por flechas que indican el flujo de información.
¿Se utiliza en análisis estructurado?	Sí, como punto de partida del modelado.	Sí, como herramienta central para representar el sistema en detalle.

Ejemplo Nro. 1:

Sistema de Gestión de Pedidos

Diagrama de Contexto:

- Muestra al sistema "Gestión de Pedidos" como una caja negra.
- Incluye al cliente, al administrador, y al proveedor como entidades externas.
- Muestra flujos como "Solicitud de pedido", "Confirmación", "Stock disponible".

DFD Nivel 1 (o mayor detalle):

Incluye procesos como "Registrar Pedido", "Verificar Stock", "Notificar al Proveedor".

Muestra cómo circula la información entre procesos y cómo se almacena en una base de datos.

Ambos diagramas están relacionados y comparten información, pero cumplen funciones distintas. El **diagrama de contexto** delimita el sistema, mientras que el **DFD** descompone el funcionamiento interno de dicho sistema, mostrando cómo la información se transforma y circula.

Ejemplo práctico

Sistema de pedidos

Se puede modelar un sistema de pedidos con los siguientes elementos:

- **Terminadores:** Cliente, Proveedor, Administración.
- **Procesos:** Recepción de pedidos, Verificación de stock, Generación de factura, Envío del producto.
- **Almacenes de datos:** Inventario, Historial de pedidos, Base de datos de clientes.
- **Flujos de datos:** Pedido de cliente, Datos del producto, Factura, Confirmación de envío.

Es importante observar cómo los procesos están numerados y vinculados por flujos de datos. Los almacenes actúan como puntos de entrada y salida intermedios de información.

Flujo de usuario

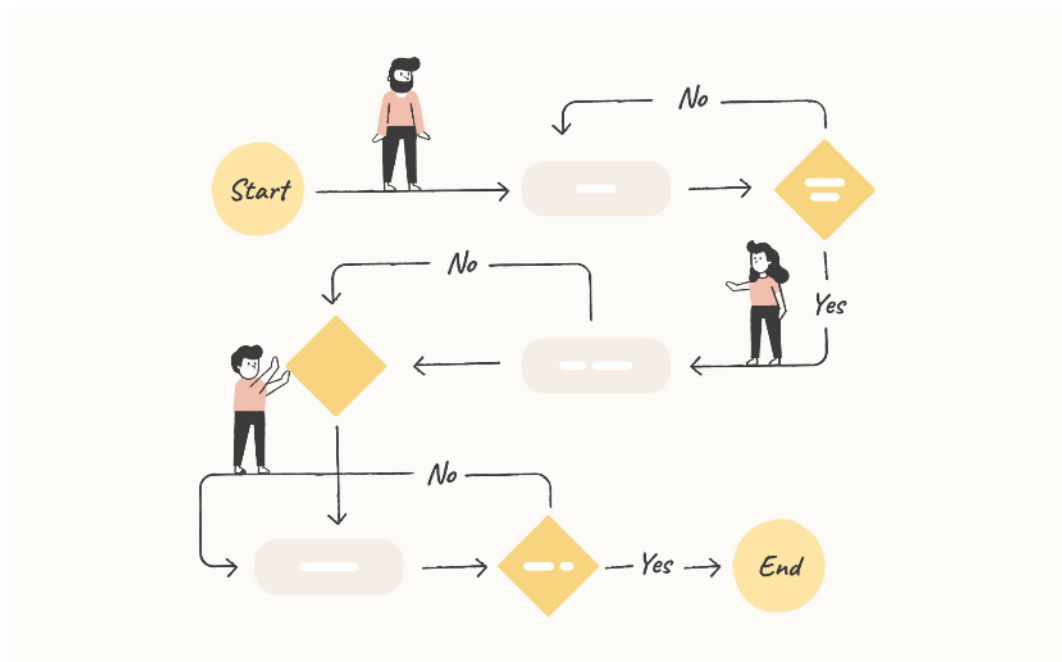


Imagen: Ilustración simple de un Flujo de Usuario

Podemos definir el flujo de usuario como la manera en que un usuario interactúa con nuestro sistema para completar tareas específicas. Es esencial tener en cuenta que este flujo se basa en cómo el usuario espera utilizar el sistema, pero también debemos considerar, como programadores, los posibles errores o flujos alternativos que el usuario podría seguir.

Definir estos flujos es crucial para limitar y programar tanto el flujo principal como los flujos alternativos válidos que el usuario puede realizar. Esto nos permite diseñar un sistema más robusto y adaptable, que responda adecuadamente a las distintas maneras en que los usuarios pueden interactuar con él.

Cuando hablamos de flujos de usuario, nos referimos a los pasos que un usuario sigue para lograr un objetivo dentro del sistema. Estos flujos deben definirse de manera clara para asegurar que el sistema sea intuitivo y cumpla con las expectativas del usuario.

Pasos para definir flujos de usuario:

Identificación del Objetivo del Usuario: comienza identificando lo que el usuario quiere lograr (por ejemplo, completar una compra, enviar un formulario, etc.).

Desglose de Tareas: divide el objetivo en tareas más pequeñas que el usuario debe realizar para alcanzar su meta (por ejemplo, seleccionar un producto, ingresar información de pago, confirmar la compra).

Creación del Flujo Principal: define el camino ideal que el usuario debería seguir para completar cada tarea de manera eficiente y sin interrupciones.

Identificación de Flujos Alternativos y Errores: considera los posibles desvíos que el usuario podría tomar, como errores de ingreso de datos o decisiones inesperadas. Define cómo el sistema debe responder en cada caso.

Validación y Ajuste: una vez definidos los flujos, pruébalos con usuarios reales o simulaciones para identificar posibles mejoras o ajustes necesarios.

Manual de usuario

Un manual de usuario de software es un documento que proporciona información detallada y orientación sobre cómo utilizar un programa o aplicación de software.

Su propósito es ayudar a los usuarios a comprender y aprovechar al máximo las funcionalidades del software, permitiéndoles utilizarlo de manera efectiva y eficiente.



En esencia, un manual de usuario es una guía que facilita a los usuarios la navegación por el software, la realización de tareas específicas y la resolución de problemas comunes que puedan surgir durante el uso.

Contiene instrucciones paso a paso, explicaciones claras y ejemplos relevantes que permiten a los usuarios aprender a utilizar el software de manera autónoma.



¿Qué debería contener un manual de usuario?

Introducción al software: una descripción general del propósito y las capacidades del software.

Instalación y configuración: instrucciones detalladas sobre cómo instalar el software en diferentes sistemas operativos y cómo configurar cualquier opción relevante.

Interfaz de usuario: una guía sobre cómo navegar y utilizar las diferentes partes de la interfaz, incluyendo menús, botones, barras de herramientas, etc.

Funcionalidades principales: instrucciones paso a paso sobre cómo realizar las tareas más comunes que el software permite, junto con ejemplos concretos.

Casos de uso: escenarios de uso detallados que muestran cómo aplicar el software en situaciones reales.

Personalización: cómo ajustar preferencias y configuraciones para adaptar el software a las necesidades del usuario.

Resolución de problemas: consejos para solucionar problemas comunes que los usuarios puedan encontrar durante el uso del software.

Preguntas frecuentes: respuestas a preguntas comunes que los usuarios podrían tener.

Contacto de soporte: información de contacto para el soporte técnico en caso de problemas más complejos.

En resumen, un manual de usuario es una herramienta esencial para que los usuarios adquieran las habilidades y el conocimiento necesarios para aprovechar al máximo el software.

Un buen manual de usuario no solo mejora la experiencia del usuario, sino que también puede reducir la necesidad de asistencia técnica y consultas de soporte, ahorrando tiempo y recursos tanto para los usuarios como para el equipo de desarrollo.



iPython!

En esta ocasión, trataremos sobre cadenas, la librería "random" y los bucles dentro de este lenguaje de Programación.

Verificar si una cadena está vacía

Opción 1: Usando "not"

La palabra clave not se usa para negar una condición. Si una cadena está vacía (""), evaluarla con **not** da como resultado True. Es decir, si dentro de un condicional uso el "**not**" puedo decir que no tiene información o está vacío "".

```

1  cadena = ""
2
3  ∨ if not cadena:
4      print("La cadena está vacía.")
5  ∨ else:
6      print("La cadena no está vacía.")

```

¿Por qué funciona?

En Python, una cadena vacía se considera falsa al evaluarse como condición. Entonces, **not cadena** será True si **cadena** está vacía.

Opción 2: Usando comillas simples ""

Otra manera que tenemos en Python para verificar si una cadena está vacía es utilizando las comillas simples "".

```

12  if cadena == "":
13      print("La cadena está vacía.")
14  else:
15      print("La cadena no está vacía.")

```

Método para invertir cadena

Una función más que les puede resultar útil es poder invertir una palabra. Python nos da la posibilidad de invertir una cadena utilizando **slicing** (`[::-1]`), una técnica muy útil en programación.

```

18  #Invertir cadena
19
20  cadena = "hola"
21  cadena_invertida = cadena[::-1]
22  print(cadena_invertida)  # Salida: "aloh"

```

Obtener longitud de una Cadena

Para saber la cantidad de caracteres de una cadena podemos utilizar también la función **len()**, nos devuelve la cantidad de caracteres de un string.

```
26  cadena = ""
27
28  if len(cadena) == 0:
29      print("La cadena está vacía.")
30  else:
31      print("La cadena no está vacía.")
```

Fíjense cómo comparamos la cadena con un 0 (cero), ya que el valor devuelto por **len()** es un entero.

Valores aleatorios en Python

Para trabajar con valores aleatorios en Python, usamos la librería **random**. Hay varias funciones útiles, veamos algunas de ellas:

1. **randrange(par1, par2)**

Esta función puede recibir dos parámetros que nos sirven para indicar el intervalo de "desde" y "hasta" en definitiva, veamos el ejemplo:

randrange(stop)

Devuelve un número entero aleatorio desde **0** hasta **stop - 1**.

```
import random

print(random.randrange(10)) # Números entre 0 y 9
```

Podemos poner un mínimo también dentro del **random.randrange**

randrange(start, stop)

```
import random

print(random.randrange(10, 20)) # Números entre 10 y 20
```

2. choice([lista])

Devuelve un entero aleatorio de una lista especificada:

```
import random

print(random.choice(["PHP", "Java", "Python"]))
```

Bucles en python

Los bucles permiten repetir una acción varias veces. En Python tenemos principalmente dos tipos de bucles:

Tipo de Bucle	¿Cuándo usarlo?
for	Cuando sabemos la cantidad de veces que se repite.
while	Cuando no sabemos la cantidad de veces, pero tenemos una condición lógica.

Los bucles en Python siguen la misma lógica que en cualquier otro lenguaje de programación. Para comprender mejor este tema, es importante entender que los bucles en cualquier lenguaje tienen una sintaxis específica y una utilidad definida. Veamos un ejemplo:

Si tengo determinada la cantidad de veces que se va a repetir una acción, probablemente estoy hablando de un bucle específico.

Sí sé que el bucle puede repetirse un número indeterminado de veces, entonces existe otro tipo de bucle adecuado para este caso.

For range(stop)

El for en Python funciona de manera similar a otros lenguajes de programación, con una diferencia clave: en Python, la estructura **for** suele usar la función **range**.

La función **range** permite definir un rango de valores sobre el cual se iterará, sin necesidad de utilizar operadores como mayor o menor (><).

Para utilizar range correctamente, tenemos que tener en cuenta que puede recibir tres parámetros:

range(start, stop, step)

Veamos cómo podemos usarlo:

1. range(start):

```
1  for i in range(5):  
2      print(i) # Imprime del 0 al 4
```

En el ejemplo anterior, la iteración realmente comienza en 0 y termina en 4, lo que sigue siendo un total de 5 elementos, como es habitual. Si utilizamos otra variación de la función range, podemos pasarle dos valores en lugar de uno, lo que indicará el inicio y el final del rango.

2. range(start, stop):

```
for i in range(2, 6):  
    print(i) # Imprime 2, 3, 4, 5
```

Otra manera de utilizar el for es aprovechando la tercera forma de la función range, que permite iterar en incrementos definidos, como "**de dos en dos**". El parámetro step indica el salto entre cada iteración.

3. range(start, stop, step)

```
for i in range(1, 10, 2):  
    print(i) # Imprime 1, 3, 5, 7, 9
```

Uso de break y else en bucles

Uso de break con else en un bucle:

En Python, podés usar "else" luego de un bucle. Esta parte solo se ejecuta si el bucle no se interrumpe con break. Si el bucle termina de forma natural (sin que se ejecute un break), se ejecuta el bloque else. Si se ejecuta un "break", el bloque "else" se omite.

Ejemplo:

```
for i in range(5):  
    print(i)  
    if i == 3:  
        break  
else:  
    print("El bucle terminó normalmente.") # Esto no se ejecuta
```

While

El while en python es una estructura que tiene en cuenta una condición determinada. Respecto al uso del while. Sabemos que existen dos tipos de while, uno es el while "normal", y otro es el do while, dentro de python no existe el do while. Pero veremos cómo podemos imitar su lógica.

El bucle **while** repite una acción mientras se cumpla una condición.

```
contador = 0  
  
while contador < 5:  
    print(contador)  
    contador += 1
```

A diferencia de otros lenguajes, en Python necesitamos utilizar "**else**" para ejecutar un bloque de código cuando el bucle haya terminado.

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1
else:
    print("El bucle terminó sin interrupciones.")
```

¿Y el do while en Python?

Cómo anticipamos, Python no tiene una estructura do while como otros lenguajes. Pero podemos simularlo:

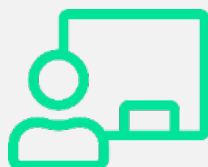
```
#Simular Do While en python
while True:
    entrada = input("Escribí 'salir' para terminar: ")
    if entrada == "salir":
        break
```

Este código siempre se ejecuta al menos una vez, como lo haría un do while.



Hemos llegado así al final de esta clase en la que vimos:

- ¿Qué es un Evento?
- Tipos de Evento que existen.
- Qué es un Diagrama de Flujo de Datos (DFD).
- Diferencias entre el Diagrama de Contexto y el DFD.
- La importancia de la Experiencia de Usuario (UX).
- Condicionales, manipulación de cadenas y bucles en Python.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.
¡Hasta la próxima clase!

Bibliografía

Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson.
https://gc.scalahed.com/recursos/files/r161r/w25469w/ingdelsoftwar elibro9_compressed.pdf

Lucid Software. (s.f.). Lucidchart pricing.
<https://lucid.app/es/pricing/lucidchart>

Diagrams.net. (s.f.). Diagrams.net. <https://app.diagrams.net>

StarUML. (s.f.). Download StarUML. <https://staruml.io/download/>

Autor desconocido. (2013, 12 de noviembre). Componentes del diagrama de flujo de datos. Sistemas de Información ITEC.
<https://sistemasdeinformacion-itec.blogspot.com/2013/11/diagrama-de-fluos-de-datos.html>

DeMarco, T. (1979). Structured analysis and system specification. Prentice Hall.

Gane, C., & Sarson, T. (1979). Structured systems analysis: Tools and techniques. Prentice Hall.

Yourdon, E. (1989). Modern structured analysis. Prentice Hall.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). Designing the user interface: Strategies for effective human-computer interaction (6th ed.). Pearson.

Python Software Foundation. (s.f.). The Python tutorial. Python.org.
<https://docs.python.org/3/tutorial>